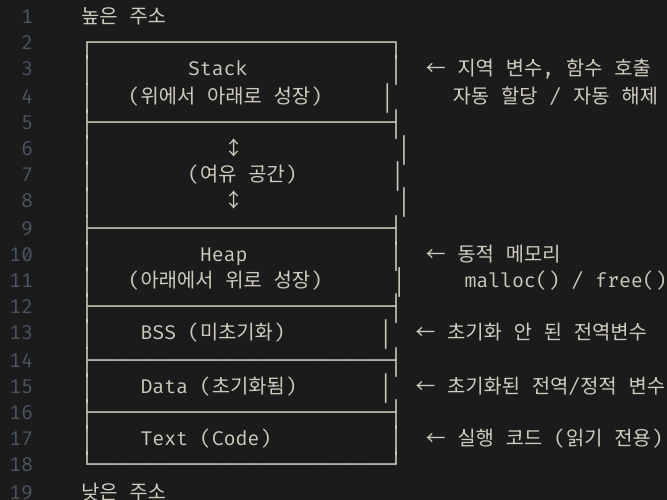


C 프로그래밍

동적 메모리 할당

프로그램의 메모리 구조

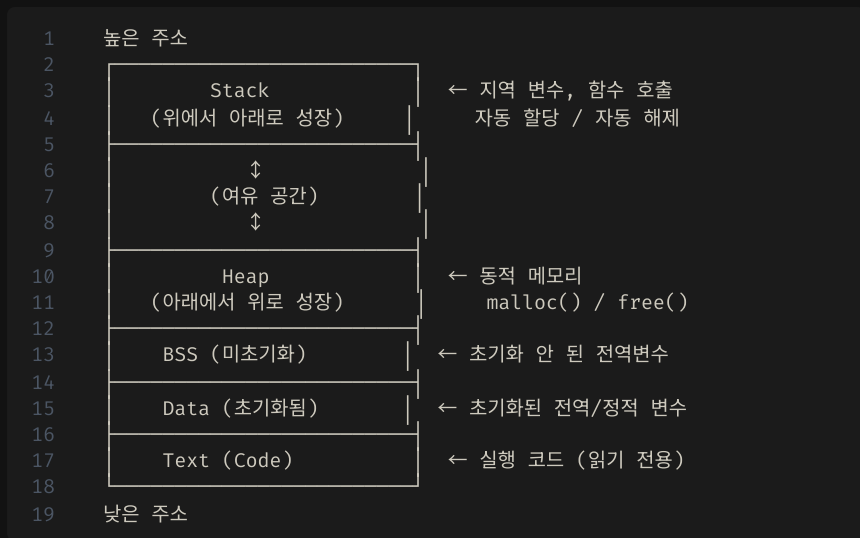
프로그램 실행 시 메모리는 크게 4개의 영역으로 나뉜다.



```
1  #include <stdlib.h>    // 동적 할당을 위한 각종 함수 제공
2
3  // Text 영역: 함수 코드
4  int add(int a, int b) { return a + b; }
5
6  // Data 영역: 초기화된 전역변수
7  int global_init = 100;
8  // BSS 영역: 초기화 안 된 전역변수
9  int global_uninit;
10
11 // Stack 영역: 지역변수 (함수 호출 시)
12 void func(void) {
13     int local = 42;      // Stack에 할당
14     int arr[100];        // Stack에 할당 (400 bytes)
15     // 함수 반환 시 자동 해제
16 }
17
18 // Heap 영역: 동적 할당
19 int* p = (int*)malloc(sizeof(int));
20 *p = 42;
21 free(p);    // ← 반드시 해제!
22 p = NULL;
```

프로그램의 메모리 구조

프로그램 실행 시 메모리는 크게 4개의 영역으로 나뉜다.



영역	설명
코드 (Code)	실행할 기계어 명령이 저장되는 영역
데이터 (Data)	전역 변수, 정적 변수가 저장되는 영역
스택 (Stack)	함수 호출 시 지역 변수.매개변수가 저장 되는 영역 (자동 관리)
힙(Heap)	프로그래머가 직접 할당·해제하는 영역 (동적 할당)

동적 메모리 할당은 힙(Heap) 영역을 사용한다.

정적 할당 vs 동적 할당

정적 할당

```
int arr[10]; // 크기를 컴파일 시점에 결정
```

- 크기가 고정됨
- 스택에 저장, 자동 해제
- 실행 전에 크기를 알아야 함

동적 할당

```
#include <stdlib.h> // 동적 할당을 위한 각종 함수 제공

int n = 10;
int* arr = (int*)malloc(n * sizeof(int));

free(arr);
```

- 실행 시점에 크기 결정 가능
- 힙에 저장, 수동 해제 필요
- 크기를 유연하게 조절 가능

동적 메모리 할당

- 동적 메모리 할당은 바이트(byte) 단위로 할당됨.
- 할당된 메모리는 타입이 없음
- 포인터 타입이 할당된 메모리를 어떻게 쓰는지 결정함.
- 프로그램이 구동된 후에야 메모리의 크기가 결정될 때만, 동적 메모리 할당을 쓰도록 하자!

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      int arr[100]; // 100개 정도는 스택에 잡으세요.
7      int* s = (int*)malloc(sizeof(int)*100); // 이런 코드는 쓰지 말 것!!!
8
9      void* p = malloc(8000);
10
11     char* pc = (char*) p;
12     int* pi = (int*) p;
13     float* pf = (float*) p;
14     double* pd = (double*) p;
15
16     free(s);
17     free(p);
18
19     return 0;
20 }
```

동적 메모리 할당 & 메모리 오류

올바른 동적 메모리 사용

```
1  #include <stdio.h>
2  #include <stdlib.h>    // 동적 할당을 위한 각종 함수 제공
3
4  int main(void) {
5      int n = 5;
6      int* arr = (int*)malloc(n * sizeof(int));
7
8      // ✅ NULL 체크 필수
9      if (arr == NULL) {
10         fprintf(stderr, "메모리 할당 실패\n");
11         return 1;
12     }
13
14     for (int i = 0; i < n; i++)
15         arr[i] = (i + 1) * 10;
16
17     for (int i = 0; i < n; i++)
18         printf("%d ", arr[i]); // 10 20 30 40 50
19
20     free(arr);    // ✅ 반드시 해제
21     arr = NULL;   // ✅ 댕글링 포인터 방지
22     return 0;
23 }
```

흔한 메모리 오류

1. 메모리 누수 (Memory Leak)

```
1  int* p = malloc(100);
2  // free(p) 빠뜨림 → 프로그램이 계속 메모리 점유
```

2. 댕글링 포인터 (Dangling Pointer)

```
1  int* p = malloc(sizeof(int));
2  free(p);
3  *p = 42; // ❌ 해제된 메모리 접근! 위험!
```

3. 버퍼 오버플로우

```
1  char buf[5];
2  strcpy(buf, "Hello, World!"); // ❌ 넘침!
3  // 보안 취약점의 주요 원인
```

4. 이중 해제 (Double Free)

```
1  free(p);
2  free(p); // ❌ 이미 해제된 메모리 재해제
```

malloc() — 메모리 할당

malloc 은 지정한 바이트 수만큼 힙에 메모리를 할당하고, 그 시작 주소를 반환한다.

malloc() 함수

```
void* malloc(size_t size);
```

- 할당 성공 시: 할당된 메모리의 포인터 반환
- 할당 실패 시: `NULL` 반환
- 할당된 메모리의 초기값은 쓰레기 값

⚠ malloc 반환값의 자료형은 `void*` 이다. 왜 그럴까?

```
// int 하나를 위한 공간 할당
int* p = (int*)malloc(sizeof(int));
```

```
// int 배열 n개를 위한 공간 할당
int* arr = (int*)malloc(n * sizeof(int));
```

malloc() 예제

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(){
5      int n;
6      printf("정수 개수 입력: ");
7      scanf("%d", &n);
8
9      // n개의 int를 힙에 할당
10     int* arr = (int*)malloc(n * sizeof(int));
11     if (arr == NULL) {
12         fprintf(stderr, "메모리 할당 실패\n");
13         return 1;
14     }
15
16     for (int i = 0; i < n; i++) {
17         arr[i] = (i + 1) * 10;
18     }
19     for (int i = 0; i < n; i++) {
20         printf("arr[%d] = %d\n", i, arr[i]);
21     }
22
23     free(arr); // 반드시 해제
24     return 0;
25 }
```

calloc() — 0으로 초기화된 할당

calloc 은 메모리를 할당하면서 모든 바이트를 0으로 초기화한다.

calloc() 함수

```
void* calloc(size_t nmemb, size_t size);
```

- `nmemb` : 요소의 개수
- `size` : 요소 하나의 바이트 크기
- `malloc(nmemb * size)` 와 같지만 초기화가 다름

calloc() 예제

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(){
5      int n = 5;
6
7      // calloc: 할당과 동시에 0으로 초기화
8      int* arr = (int*)calloc(n, sizeof(int));
9      if (arr == NULL) {
10         fprintf(stderr, "메모리 할당 실패\n");
11         return 1;
12     }
13
14     // 초기값 확인 (모두 0)
15     for (int i = 0; i < n; i++) {
16         printf("arr[%d] = %d\n", i, arr[i]);
17     }
18
19     free(arr);
20     return 0;
21 }
```


free() — 메모리 해제

동적 할당한 메모리는 사용 후 반드시 `free()` 로 해제해야 한다.

free() 함수

```
void free(void* ptr);
```

- `malloc`, `calloc` 으로 할당한 포인터만 전달
- 같은 포인터를 두 번 해제하면 정의되지 않은 동작
- 해제 후 포인터를 `NULL` 로 설정하는 것이 안전

```
free(arr);  
arr = NULL; // 댕글링 포인터 방지
```

메모리 누수와 댕글링 포인터

```
1  #include <stdio.h>  
2  #include <stdlib.h>  
3  
4  // ❌ 나쁜 예: 메모리 누수  
5  void bad_example() {  
6      int* p = (int*)malloc(sizeof(int));  
7      *p = 42;  
8      // free(p) 호출 누락 → 메모리 누수 발생  
9  }  
10  
11 // ✅ 좋은 예: 올바른 해제  
12 void good_example() {  
13     int* p = (int*)malloc(sizeof(int));  
14     if (p == NULL) return;  
15     *p = 42;  
16     printf("값: %d\n", *p);  
17     free(p); // 반드시 해제  
18     p = NULL; // 댕글링 포인터 방지  
19 }  
20  
21 int main() {  
22     bad_example(); // 누수 발생  
23     good_example(); // 정상  
24     return 0;  
25 }
```

free() — 메모리 해제

동적 할당된 메모리는 사용 후 반드시 `free()` 로 해제해야 한다.

주의사항 정리

문제	원인	해결책
메모리 누수	<code>free()</code> 미호출	모든 경로에서 <code>free()</code> 호출
댄글링 포인터	<code>free()</code> 후 포인터 사용	<code>free()</code> 후 <code>p = NULL</code>
이중 해제	같은 포인터 두 번 <code>free()</code>	<code>free()</code> 후 <code>p = NULL</code>
NULL 역참조	할당 실패 확인 누락	항상 <code>NULL</code> 체크
버퍼 오버플로	할당 범위 초과 접근	인덱스 범위 확인

메모리 크기 재조정 — malloc() + memcpy() + free()

새 블록을 할당하고 데이터를 복사한 뒤 기존 블록을 해제한다.

패턴

```
#include <string.h> // memcpy

int new_cap = old_cap * 2;
int* tmp = (int*)malloc(new_cap * sizeof(int));
if (tmp == NULL) { free(arr); return 1; }
memcpy(tmp, arr, old_cap * sizeof(int));
free(arr);
arr = tmp;
```

단계	함수	역할
1	malloc	새 크기만큼 힙 공간 확보
2	memcpy	기존 데이터를 새 공간으로 복사
3	free	기존 블록 해제

예제

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  int main() {
6      int old_cap = 3;
7      int* arr = (int*)malloc(old_cap * sizeof(int));
8      if (arr == NULL) return 1;
9
10     arr[0] = 10; arr[1] = 20; arr[2] = 30;
11     printf("확장 전: %d %d %d\n", arr[0], arr[1], arr[2]);
12
13     // 크기를 3 → 6으로 확장
14     int new_cap = 6;
15     int* tmp = (int*)malloc(new_cap * sizeof(int));
16     if (tmp == NULL) { free(arr); return 1; }
17     memcpy(tmp, arr, old_cap * sizeof(int));
18     free(arr);
19     arr = tmp;
20
21     arr[3] = 40; arr[4] = 50; arr[5] = 60;
22     for (int i = 0; i < new_cap; i++) {
23         printf("arr[%d] = %d\n", i, arr[i]);
24     }
25
26     free(arr);
27     return 0;
28 }
```

요약

```
#include <stdlib.h>    // 동적 할당을 위한 각종 함수 제공
```

함수	역할	초기값
<code>malloc(size)</code>	<code>size</code> 바이트 할당	쓰레기 값
<code>calloc(n, size)</code>	<code>n * size</code> 바이트 할당	0
<code>free(ptr)</code>	할당된 메모리 해제	—
<code>memcpy(dst, src, n)</code>	<code>n</code> 바이트 복사 (<code><string.h></code>)	—

동적 할당을 사용하면 유연한 자료구조를 만들 수 있지만, 메모리 관리 책임은 전적으로 프로그래머에게 있다.